LÖSUNGEN · LEHREREXEMPLAR

W2·3 – Effizienz und binäre Suche: Lösungen

Zum Arbeitsblatt `AQ_W2_AB3_Effizienz_und_binaere_Suche.docx` · Code: `AQ_W2_SortierDuell_LSG.txt`, `AQ_W2_L2_Rangliste_LSG.txt` · alle Messwerte mit einem Java-Nachbau der Klasse Vis (gleicher Pseudozufall) nachgerechnet

Aufgabe 0 – Zähl-Warm-up

Punkte	Vergleiche	Vertauschungen	Vergleiche bei selectionSort
40, 30, 20, 10 (bester Fall)	3	0	6
10, 20, 30, 40 (schlechtester Fall)	6	6	6
40, 10, 30, 20	5	2	6

Selectionsort vergleicht immer 3 + 2 + 1 = 6-mal – Gesprächsanlass für Aufgabe 1 f).

Aufgabe 1 – Messreihe mit der Visualisierung

a) Bubblesort: Große Balken wandern Schritt für Schritt (immer nur zum Nachbarn) nach rechts; rechts wächst der fertige Bereich. **b)** Selectionsort: Ein Suchlauf über den Rest, dann **ein** weiter Sprung-Tausch; links wächst der fertige Bereich. Insertionsort: Links wächst ein sortierter Block, aus dem rechten Teil wandert jeweils ein Balken nach links hinein. Code: `AQ_W2_SortierDuell_LSG.txt` (Training 1 und 2).

c) Messwerte (Vergleiche; reproduzierbar, da fester Startwert):

n	Bubblesort (Vorlage) zufall	Selectionsort zufall	Insertionsort zufall	Insertionsort umgekehrt	n(n − 1)/2
10	45	45	31	45	45
20	190	190	108	190	190
40	780	780	391	780	780
80	3160	3160	1629	3160	3160

d) Insertionsort (zufall): 391 / 108 ≈ **3,6** und 1629 / 391 ≈ **4,2**. Bubble- und Selectionsort: 780 / 190 ≈ 4,1 und 3160 / 780 ≈ 4,05. **Regel:** Verdoppelt man n, vervierfacht sich die Zahl der Vergleiche ungefähr. (Bei zufälligen Daten schwanken die Insertionsort-Werte um n(n − 1)/4.)

e) Training 3 (n = 40; Vergleiche, in Klammern Vertauschungen):

Verfahren	zufall	fast sortiert	umgekehrt	viele gleiche	Summe
Bubblesort (Vorlage)	780 (356)	780 (40)	780 (780)	780 (325)	**3120** (1501)
Selectionsort	780 (34)	780 (4)	780 (20)	780 (30)	**3120** (88)
Insertionsort	391 (356)	79 (40)	780 (780)	362 (325)	**1612** (1501)

Insertionsort gewinnt überall außer bei „umgekehrt“ (Gleichstand) und ist auf „fast sortiert“ unschlagbar billig. Selectionsort tauscht mit Abstand am wenigsten. Bubble- und Insertionsort tauschen gleich oft (Zahl der Fehlstände).

f) Selectionsort: Die innere Schleife durchsucht in jedem Durchlauf den **ganzen** Rest, es gibt keinen vorzeitigen Abbruch – also immer (n − 1) + … + 1 = n(n − 1)/2. Die Bubblesort-**Vorlage** hat ebenfalls keinen Abbruch: Sie macht alle n − 1 Durchläufe, auch wenn längst sortiert ist. Dein bubbleSort aus Level 2.2 bricht nach einem Durchlauf ohne Vertauschung ab – auf sortierten Daten nach n − 1 Vergleichen.

Aufgabe 2 – Nachrechnen statt messen (12 BE)

a) 1. Durchlauf: n − 1 Vergleiche, 2. Durchlauf: n − 2, letzter Durchlauf: 1. Summe S = (n − 1) + (n − 2) + … + 1. Rückwärts geschrieben: S = 1 + 2 + … + (n − 1). Spaltenweise addiert ergibt jede der n − 1 Spalten n, also 2S = n(n − 1) und **S = n(n − 1)/2** (3 BE).

b) n = 40: **780**; n = 80: **3160**; n = 1000: **499 500** – stimmt exakt mit den Messungen für Selection- und Bubblesort überein (1 BE).

c) V(2n) = 2n(2n − 1)/2 = n(2n − 1). Verhältnis: n(2n − 1) / (n(n − 1)/2) = (4n − 2)/(n − 1) – für große n fast genau 4 (n = 40: 3160/780 ≈ 4,05). **Satz:** *Verdoppelt man die Anzahl der Einträge, so vervierfacht sich die Anzahl der Vergleiche ungefähr, da sie quadratisch von n abhängt.* (3 BE)

d) Bester Fall: **n − 1** Vergleiche, z. B. 50, 40, 30, 20, 10 (schon absteigend) → 4 Vergleiche, 0 Vertauschungen. Schlechtester Fall: **n(n − 1)/2**, z. B. 10, 20, 30, 40, 50 (aufsteigend) → 10 Vergleiche und 10 Vertauschungen. Im besten Fall **verdoppelt** sich der Aufwand bei Verdopplung von n nur (3 BE).

e) z. B. **20, 50, 40, 30, 10**: Im 1. Durchlauf „sinkt" 20 bis vor die 10 → 50, 40, 30, 20, 10; der 2. Durchlauf tauscht nicht mehr → Ende nach 4 + 3 = **7 Vergleichen** (3 Vertauschungen). Auch 50, 40, 30, 10, 20 (7 Vergleiche, 1 Vertauschung). Idee: Ein schlechter Eintrag wandert in einem Durchlauf beliebig weit nach hinten, ein guter nur einen Platz nach vorne (2 BE).

Aufgabe 3 – Sortier-Duell (nur für die Lehrkraft)

Referenzwerte – NICHT an Schüler weitergeben

Summe der Vergleiche über die vier Runden (je 40 Balken), in Klammern Vertauschungen; mit dem Gerüst reproduzierbar (Lehrerwerkzeug `Referenz.tabelle()` in der LSG-Datei):

Verfahren	zufall	fast sort.	umgekehrt	viele gl.	Summe
Bubblesort (Vorlage)	780	780	780	780	3120 (1501)
Bubblesort mit Abbruch	770	504	780	752	2806 (1501)
Selectionsort	780	780	780	780	3120 (88)
Insertionsort (Standard)	391	79	780	362	1612 (1501)
Quicksort, Pivot = letztes	188	456	780	208	1632 (173)
binäres Einfügen (LSG)	158	150	177	154	**639** (1501)

Einordnung: 3120 = unverändert Bubble/Selection · um 2800 = Bubblesort mit Abbruch · **1612** = Ausgangswert des Gerüsts (a) · 640 – 1600 Teilverbesserungen · um **640** = binäres Einfügen (Musterlösung) · unter 600 = adaptive Ideen (z. B. umgekehrte Daten erkennen) – unbedingt vorstellen lassen. Untere Schranke zur Einordnung: Kein vergleichsbasiertes Verfahren sortiert **alle** Anordnungen von 40 Werten mit weniger als $\log_2(40!) \approx 160$ Vergleichen.

b) Insertionsort sucht die Einfügestelle **linear** im schon sortierten linken Teil – dort könnte man **binär** suchen (Brücke zu Teil B). Weitere Ideen: Ausgangslage erkennen (z. B. „umgekehrt" zuerst prüfen), Vorab-Test mit dem linken Nachbarn (lohnt sich nur bei fast sortierten Daten: 663 statt 639 in der Summe – „messen statt glauben").

```
static void duellSort() {                  // binaeres Einfuegen (LSG)
    for (int i = 1; i < Vis.laenge(); i++) {
        int lo = 0;
        int hi = i;
        while (lo < hi) {                      // Stelle in 0 .. i suchen
            int mid = (lo + hi) / 2;
            if (Vis.kleiner(i, mid)) {
                hi = mid;
            } else {
                lo = mid + 1;                  // gleiche Werte: dahinter (stabil)
            }
        }
        for (int j = i; j > lo; j--) {         // nur Nachbartausche moeglich
            Vis.vertausche(j, j - 1);
        }
    }
}
```

Auswertung im Unterricht: Ergebnisse der Teams an der Tafel bzw. am Beamer sammeln – Wert = Vergleiche (weniger ist besser), Tiebreak = Vertauschungen, gültig ja/nein. Ergebnisse mit gueltig = nein werden nicht gewertet. Das Duell ist ein Unterrichtsspiel ohne Lab-Wertung; Punkte gibt es nur im Lab (Lab-Level 2.3).

Aufgabe 4 – Zahlenraten

b) Immer die Mitte des noch möglichen Bereichs raten – jede Antwort halbiert den Bereich: 100 → 50 → 25 → 12 → 6 → 3 → 1. **Höchstens 7** Versuche ($2^7 = 128 \geq 100$). **c)** 1 bis 200: **8** Versuche; 1 bis 1000: **10** Versuche ($2^{10} = 1024$).

Aufgabe 5 – Trace lo, hi, mid (7 BE)

a) x = 820:

Durchlauf	lo	hi	mid	r[mid]	Aktion
1	0	9	4	610	lo ← 5
2	5	9	7	905	hi ← 6
3	5	6	5	750	lo ← 6
4	6	6	6	820	gib 6 zurück

Rückgabewert **6**, **4** Durchläufe (3 BE). **b)** x = 700:

Durchlauf	lo	hi	mid	r[mid]	Aktion
1	0	9	4	610	lo ← 5
2	5	9	7	905	hi ← 6
3	5	6	5	750	hi ← 4
–	5	4			gib −1 zurück

Rückgabewert **−1** nach **3** Durchläufen, Ende mit **lo = 5, hi = 4** – lo zeigt auf die Stelle, an der 700 eingefügt werden müsste (3 BE). **c)** Linear: 820 nach **7** geprüften Elementen, für 700 alle **10** (1 BE). Ausgaben des Hauptprogramms: 820 linear Index 6 nach 7, binär Index 6 nach 4, 700 binär −1 nach 3, Einfügeposition für 700: 5.

Aufgabe 6 – Wie oft höchstens? (12 BE)

nach Durchlauf	0	1	2	3	4	5	6	7
höchstens noch … Elemente	100	50	25	12	6	3	1	0

a) Ein erfolgloser Durchlauf schließt mid und eine Hälfte aus: Von s Elementen bleiben höchstens ⌊s/2⌋ übrig (bei gerader Anzahl ist die rechte Hälfte die größere). Nach 7 Durchläufen ist der Bereich leer (lo > hi) – spätestens dann endet die Schleife; ein Treffer beendet sie früher (3 BE).

n	1	2	4	8	16	100	200	1000	1 000 000
max. Durchläufe	1	2	3	4	5	7	8	10	20

b) siehe Tabelle; allgemein $\lfloor \log_2 n \rfloor + 1$ – alle Werte mit einer Simulation über alle vorhandenen und fehlenden Werte geprüft (2 BE). **c)** Nach dem ersten Durchlauf bei 2n Elementen bleiben höchstens n übrig – ab da verläuft die Suche wie bei n Elementen. Also höchstens **ein** Durchlauf mehr (2 BE).

n	lineare Suche	binäre Suche	Faktor
1 000	1 000	10	100
1 000 000	1 000 000	20	50 000

d) siehe Tabelle (2 BE). **e) Erwartung (3 BE):** Sortieren mit Insertionsort kostet im schlechtesten Fall $n(n-1)/2 \approx \mathbf{5 \cdot 10^{11}}$ Vergleiche. Eine lineare Suche kostet höchstens 10^6, 10 000 Suchen höchstens 10^{10}. Sortieren + 10 000 binäre Suchen (je ≤ 20) ≈ $5 \cdot 10^{11} + 2 \cdot 10^5$. **In beiden Fällen lohnt sich das Sortieren mit Insertionsort nicht** – erst ab etwa 500 000 Suchen. Mit einem schnelleren Sortierverfahren (Mergesort, Welt 3: etwa $n \cdot \log_2 n \approx 2 \cdot 10^7$ Vergleiche) lohnt es sich schon ab ungefähr 20 Suchen. Bewertet wird die begründete Abwägung mit Zahlen, nicht das genaue Ergebnis.

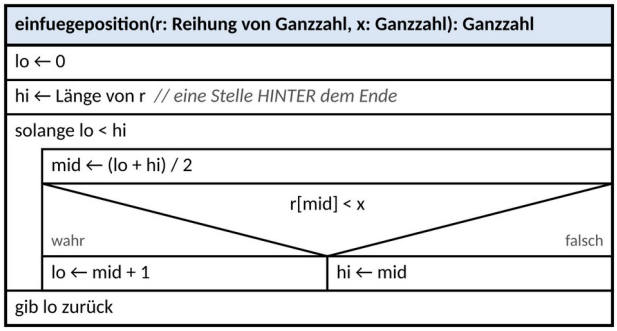
Aufgabe 7 – IDE-Level 2.3

Lösungen und typische Fehler: `AQ_W2_L2_Rangliste_LSG.txt` (linear, binaer, einfuegeposition). Häufig: durchlaeufe nicht auf 0 gesetzt (Test `erweiterung_linear_durchlaeufe`), `lo < hi` statt `lo <= hi` (Reihung {7}), `lo = mid` → Endlosschleife (Testrunner hängt – Programm stoppen).

Für Schnelle – einfuegeposition

Antwort auf den Tipp: Ist r[mid] ≥ x, kann mid selbst der gesuchte kleinste Index sein – hi = mid − 1 würde ihn verlieren. Darum hi = mid und Schleife mit lo < hi (sonst Endlosschleife bei lo = hi). Probe: r = {10, 20, 20, 30}: x = 20 → 1, x = 35 → 4; Punktetabelle: x = 700 → 5. **Denkanstoß:** Genau diese Suche steckt im LSG-duellSort (binäres Einfügen) – sie senkt die Summe von 1612 auf 639 Vergleiche.

```
public static int einfuegeposition(int[] r, int x) {
    durchlaeufe = 0;
    int lo = 0;
    int hi = r.length;            // Antwort liegt immer in lo .. hi
    while (lo < hi) {
        durchlaeufe++;
        int mid = (lo + hi) / 2;
        if (r[mid] < x) {
            lo = mid + 1;         // mid ist zu klein: Antwort rechts davon
        } else {
            hi = mid;             // mid kommt selbst in Frage
        }
    }
    return lo;
}
```



Teil C – Speicherbedarf (nur Leistungskurs, eA)

Aufgabe 8 – Wie viel Platz extra? (6 BE)

a) hoechstePunktzahl(): **konstant** – nur max und der Laufindex, unabhängig von n. selectionSort(): **konstant** – getauscht wird innerhalb der Reihung, dazu nur i, j, der Index des Besten und eine Tauschvariable. kopie(): **linear** – die neue Reihung hat anzahl = n Felder. Turniertabelle: **quadratisch** – n · n Felder (je 1 BE).

b) n = 30: 30 · 30 = **900** Felder, n = 60: **3600** Felder → Faktor **4**, denn $(2n)^2 = 4n^2$ (2 BE).

Aufgabe 9 – Zwei Detektoren beurteilen (8 BE)

a) A prüft ohne Doppel alle Paare: $n(n-1)/2 = 200 \cdot 199 / 2 =$ **19 900** Vergleiche; B fragt jeden Wert einmal ab: **200** Abfragen (2 BE; Werte mit einer Java-Fassung beider Verfahren nachgezählt).

b) A: **konstant** – keine zusätzliche Reihung, nur i und j. B: eine Reihung mit **1001** Feldern – unabhängig von n, abhängig vom Wertebereich 0 … 1000 (2 BE).

c) Erwartung (4 BE): Operationen: A wächst **quadratisch** (doppeltes n → etwa vierfache Vergleiche), B **linear** (doppeltes n → doppelte Abfragen). Speicher: A konstant, B fest 1001 Felder. **Urteil:** Für große Ranglisten mit Punkten von 0 bis 1000 ist B klar effizienter – bei n = 100 000 knapp $5 \cdot 10^9$ Vergleiche gegenüber 100 000 Abfragen, und 1001 Felder sind wenig. Bei sehr kleinem n spielt der Unterschied kaum eine Rolle (n = 10: höchstens 45 Vergleiche). Sind beliebige int-Werte erlaubt, bräuchte B rund 4,3 Milliarden Felder (2^{32}) – dann ist B ungeeignet, A (oder erst sortieren und Nachbarn vergleichen, Welt 3) ist vorzuziehen. Kernpunkte: beide Größen genannt · Abschätzung abhängig von n · Vergleich · situationsbezogenes, begründetes Urteil inkl. Wertebereich (je 1 BE).

Typische Fehler: nur „B ist schneller“ ohne Speicher (P10: kein Urteil) · „B braucht n Felder“ (die Strichliste hängt vom Wertebereich ab) · Selectionsort wird zusätzlicher Speicher zugeschrieben.